

Material de Aula

Curso: Bacharelado em Ciência da Computação

Disciplina: Tecnologia de Orientação a Objetos (TOO)

Professora: Vanessa Lago Machado

Programação Orientada à Objetos (POO)

10. Pilar Polimorfismo

10.1. Conceito de Polimorfismo

A palavra **Polimorfismo** vem do grego e significa “**muitas formas**”.

Na **Programação Orientada a Objetos (POO)**, ele representa a **capacidade de um mesmo método ou comportamento assumir diferentes formas**, dependendo do **objeto** que o invoca.

- Em outras palavras: **objetos de classes diferentes podem responder de formas distintas ao mesmo método.**
-

◆ Exemplo intuitivo:

Imagine o método comunicar():

- Uma classe **Cachorro** pode fazer falar() → “Au au!”
- Uma classe **Gato** pode fazer falar() → “Miau!”
- Ambas possuem o **mesmo nome de método**, mas **comportamentos diferentes**.

```
class Animal(ABC):
    @abstractmethod
    def comunicar(self):
        pass

class Cachorro(Animal):
    def comunicar(self):
        return "Au au!"

class Gato(Animal):
    def comunicar(self):
        return "Miau!"

animais = [Cachorro(), Gato()]
for a in animais:
    print(a.comunicar())
```

Saída:

```
Au au!
Miau!
```

Ambos executam o mesmo método `comunicar()`, mas cada um tem seu **próprio comportamento** — isso é **polimorfismo**.

10.2. Polimorfismo em Python

Em Python, o polimorfismo pode ocorrer de **duas formas principais**:

- **Polimorfismo de Subclasses (Override)**

Ocorre quando uma **subclasse sobrescreve** um método da classe-pai.

Exemplo: `exibir_dados()` da classe `Tarefa`, sobreescrito nas classes `TarefaPessoal` e `TarefaProfissional`.

- **Polimorfismo de Funções (Duck Typing)**

Ocorre quando objetos de classes diferentes possuem o **mesmo método**, sem herança direta.

Exemplo: `obj.exibir_dados()` funciona para qualquer objeto que possua o método.

10.3. Polimorfismo com Sobrescrita de Métodos

Na herança, as classes filhas podem **sobrescrever métodos** da classe pai para **adaptar seu comportamento**.

Exemplo: Classes de Tarefas

```
class Tarefa(ABC):
    [...]

    @abstractmethod
    def exibir_dados(self):
        pass
```

Subclasse 1 – TarefaPessoal

```
from model.Tarefa import Tarefa

class TarefaPessoal(Tarefa):
    def __init__(self, nome_tarefa, tipo=None, descricao=None):
        super().__init__(nome_tarefa, descricao)
        self.tipo = tipo

    def exibir_dados(self):
        return f"[Tarefa Pessoal]\n"\
            f"- {self.nome}\n - Tipo: {self.tipo or 'Não informado'}"
```

Subclasse 2 – TarefaProfissional

```
from model.Tarefa import Tarefa

class TarefaProfissional(Tarefa):
    def __init__(self, nome_tarefa, projeto, data_entrega, descricao=None):
```

```
super().__init__(nome_tarefa, descricao)
self.projeto = projeto
self.data_entrega = data_entrega

def exibir_dados(self):
    return (f"[Tarefa Profissional]\n"
            f" - Nome: {self.nome}\n"
            f" - Projeto: {self.projeto}\n"
            f" - Entrega: {self.data_entrega}")
```

Cada classe filha (TarefaPessoal, TarefaProfissional) implementa **seu próprio comportamento** para `exibir_dados()`.

Assim, podemos tratar **todas como “Tarefas”**, mas cada uma reage de forma diferente.

10.5. Benefícios do Polimorfismo

- **Reutilização de Código**
Evita duplicação e permite reaproveitar comportamentos da classe base.
 - **Flexibilidade**
Permite tratar objetos diferentes de forma uniforme.
 - **Extensibilidade**
Facilita a adição de novas classes sem alterar o código existente.
 - **Leitura e Manutenção**
O código fica mais limpo e intuitivo.
-

10.6. Resumo Polimorfismo

- **Polimorfismo** é a habilidade de usar o mesmo método para **comportamentos diferentes**.
- Ele trabalha em conjunto com **herança** e **abstração**, promovendo **flexibilidade e reuso** no código.

- Na prática, é o que permite criar sistemas extensíveis, dinâmicos e bem estruturados.
-

Exercício Proposto – Aplicando Polimorfismo

1. Crie\ Edite a nova classe **TarefaGamer**, herdando de Tarefa.

- Implemente os **métodos abstratos** definidos na classe base.
 - Adicione pelo menos um novo atributo específico dessa classe (exemplo: jogo, nível, ou plataforma).
 - O método `exibir_dados()` deve mostrar as informações completas da tarefa, de forma coerente com os outros tipos.
-

2. Crie um programa de teste contendo uma **lista com, no mínimo, 4 tarefas**, sendo:

- Uma instância de `TarefaPessoal`
 - Uma instância de `TarefaProfissional`
 - Uma instância de `TarefaGamer`
 - E outra instância de qualquer tipo de tarefa anterior
-

3. Em algumas tarefas, altere o estado para **concluída**, utilizando o método `concluir()`.

- Ao concluir uma tarefa, **chame o método `definir_termino()`**, definindo a data e hora de finalização.